

Deadlock

Deadlock

Learning Objectives:

- Be able to explain what a deadlock is.
- Be able to read a resource-allocation graph to identify processes and resources that you could preempt to resolve deadlock.
- Be able to create a global locking order to eliminate deadlock issues.

Motivation—Dining Philosophers

- A group of N philosophers sit around a circular table.
- The philosophers want to go back and forth between:
 - Eating for some time
 - Thinking for some time
- The philosophers are eating a special dish that requires *two forks* to consume, but, there are only N forks!
- Possible solution:
 - Associate a lock with each fork.
 - Each philosopher:
 - thinks
 - acquires right fork
 - acquires left fork
 - Eats
- See [lectures/synchronization/dining.c](#) example. What happens when we run it?

Deadlock

- A deadlock occurs when a set of threads each waits for an event that only another thread in the set can cause.
 - It causes *starvation*, the program appears to stop executing